Nombre: Juan Camilo Ossa Diaz

Asignatura: Estructura de datos

Docente: Juan Guillermo Duque Galvis

Fecha: 13/09/2026

Solución Integral: Taller de Complejidad Algorítmica y Estructuras de Datos

1. Análisis de Complejidad Algorítmica (Big O)

En la arquitectura de software moderna, la notación **Big O** no representa una simple medida de tiempo cronométrico, sino que constituye el lenguaje estratégico fundamental para evaluar la eficiencia y escalabilidad de un sistema antes de comprometer recursos en su implementación. Comprender **Big O** permite al ingeniero predecir el comportamiento del software bajo condiciones de estrés hídrico de datos, asegurando que la infraestructura soporte el crecimiento sin degradaciones catastróficas del servicio.

1.1 Clasificación y Justificación de Casos

La siguiente tabla clasifica la eficiencia de diversas operaciones lógicas basadas en su tasa de crecimiento:

Situación	Complejidad	Justificación Cualitativa
Buscar palabra en diccionario físico (mitades)	**O(log n)**	La técnica de división y conquista reduce el espacio de búsqueda a la mitad en cada iteración, permitiendo que el esfuerzo crezca de forma logarítmica respecto al total de páginas.
Revisar carnés uno por uno	**O(n)**	El esfuerzo es directamente proporcional a la cantidad de elementos; cada nuevo carné añade una unidad de tiempo constante al proceso total.
Sacar la primera carta de un mazo	**O(1)**	Es una operación de acceso directo. La cantidad total de cartas en el mazo es irrelevante para la acción de retirar el elemento superior.
Comparar estudiantes entre sí (cumpleaños)	**O(n^2)**	Requiere un procesamiento anidado: por cada estudiante nuevo, se deben realizar comparaciones con todos los miembros

		existentes, resultando en un crecimiento cuadrático.
Consultar la hora en el reloj	**O(1)**	Representa un acceso a una variable de estado global de tiempo constante, independiente del contexto o de otras variables del sistema.

1.2 La Linealidad en la Vida Real

Consideremos una tarea administrativa común: **la revisión manual de facturas físicas no clasificadas para encontrar un ID específico**. Esta es una operación **O(n),** ya que el tiempo total de búsqueda depende estrictamente del volumen de documentos en el legajo.

Si la entrada **(n)** escala por un factor de **10** (pasando de **100** a 1,000 facturas), el tiempo de ejecución se multiplicará igualmente por **10.** Aunque este crecimiento es lineal y predecible, se vuelve problemático en entornos de alta transaccionalidad; un proceso que manualmente toma **10** minutos se transformaría en **100** minutos, evidenciando que las soluciones lineales requieren automatización o estructuras de búsqueda más eficientes **O(log n)** al superar ciertos umbrales críticos.

1.3 Evaluación de Escalabilidad (Forma A vs. Forma B)

Mi recomendación para un sistema con crecimiento incierto es, sin duda, la **Forma B O(log n)**

Si bien para volúmenes triviales **(n=5)** la Forma **A O(n^2)** puede presentar un tiempo de ejecución menor debido a constantes de implementación más bajas, su curva de crecimiento es insostenible. Mientras que **O(n^2)** se dispara hacia la ineficiencia ante incrementos modestos de datos, **O(log n)** mantiene una pendiente controlada. Para un arquitecto, la prioridad no es la velocidad en entornos de prueba minúsculos, sino la viabilidad del sistema cuando la carga operativa aumente en órdenes de magnitud.

1.4 Validación de Conceptos Fundamentales

1. **Un algoritmo O(1) siempre es más rápido en segundos reales que uno O(n).**
    - **Falso. Big O** mide la **tasa de crecimiento**, no el tiempo absoluto. Un proceso **O(1)** con una constante de ejecución muy alta (ej. 1 hora) será más lento que un **O(n**) simple con **n=10** que tarde milisegundos.

La superioridad del **O(1)** solo se garantiza a medida que n tiende al infinito.

2. **La notación Big O describe cómo crece el tiempo de ejecución a medida que crecen los datos, no el tiempo exacto en segundos.**
    - **Verdadero.** Es un modelo matemático que ignora factores variables como el hardware, el lenguaje o las constantes menores, centrándose exclusivamente en la tendencia de escalabilidad del algoritmo.
3. **Buscar por índice en un arreglo (arreglo[5]) es O(n), porque hay que recorrer los primeros 5 elementos.**
    - **Falso.** El acceso es **O(1).** Los arreglos utilizan aritmética de punteros para saltar directamente a la dirección de memoria deseada sin inspeccionar los elementos previos.

Comprender la eficiencia algorítmica es el requisito previo indispensable para diseñar la estructura física de los datos, comenzando por la unidad básica de almacenamiento: el arreglo.

2. Estructuras Unidimensionales: Arreglos 1D

Desde la perspectiva de bajo nivel, un arreglo **1D** es una estructura de **acceso aleatorio** cuya eficiencia reside en la contigüidad de la memoria. No es una mera lista secuencial, sino un bloque sólido en la **RAM** donde la posición de cualquier elemento puede calcularse instantáneamente mediante una operación aritmética simple.

2.1 Diseño y Acceso Indexado

Representación lógica del arreglo notas:



- **Tercer elemento:** El valor ingresado es **2.8**, ubicado en el **índice 2**.
- **Acceso al último elemento (Pseudocódigo):**

VALOR ultimo_estudiante = notas[5]

2.2 Aritmética de Memoria y Acceso O(1)

Para un arreglo con base 0x2000 y elementos de 4 bytes, el cálculo de direcciones se realiza mediante: Dirección = Base + (Índice * Tamaño)

- **notas[0]:** 0x2000 + (0 \times 4) = \mathbf {0x2000}
- **notas[3]:** 0x2000 + (3 \times 4) = 0x2000 + 12. En hexadecimal, 12 = C, resultando en **0x200C**.
- **notas[5]:** 0x2000 + (5 \times 4) = 0x2000 + 20. En hexadecimal, 20 = 0x14, resultando en **0x2014**.

El factor clave ("So What?"): La rapidez de acceso es **O(1)** porque el procesador no necesita recorrer el arreglo. El tiempo para resolver la suma y multiplicación es constante independientemente del índice. Esto solo es posible porque los arreglos son **homogéneos**: al exigir un único tipo de dato, el **"tamaño"** en la fórmula es constante, permitiendo saltos precisos en la **RAM**.

2.3 Búsqueda Lineal vs. Búsqueda Binaria

Búsqueda del valor **31** en edades = [15, 18, 20, 23, 27, 31, 35, 40]:

1. **Búsqueda Lineal:** Compara secuencialmente: 15, 18, 20, 23, 27, 31. **Total: 6 comparaciones.**
2. **Búsqueda Binaria:**
    - **Paso 1:** Mid = (0+7)/2 = 3. edades[3] es 23. Como 31 > 23, buscamos en la mitad superior.
    - **Paso 2:** Mid = (4+7)/2 = 5. edades[5] es 31. **Total: 2 comparaciones.**

La búsqueda binaria requiere orden previo porque utiliza la lógica de descarte: si el punto medio es menor que el objetivo, se garantiza que todos los elementos a la izquierda también lo son. En un escenario de **1 millón de elementos**, la búsqueda lineal podría requerir hasta un millón de comparaciones (O(n)), mientras que la binaria solo requeriría \approx 20 **O(log n).**

marcando la diferencia entre un sistema ágil y uno inoperante.

2.4 Dinámica de Inserción y Costos Operativos

- **Al final con espacio:** O(1). Operación directa de escritura en la dirección base + (n * tamaño).
- **Al final con redimensionamiento:** O(n). Requiere asignar un nuevo bloque de memoria y copiar todos los elementos existentes al nuevo destino.

- **En el medio (insertar 21):** O(n). Para mantener la contigüidad y el orden, todos los elementos desde la posición de inserción hasta el final (23, 27, etc.) deben desplazarse físicamente una casilla a la derecha.

La inserción intermedia es estructuralmente más costosa debido al **costo de desplazamiento**, una limitación física de los arreglos que el diseño de software debe considerar para evitar cuellos de botella. Esta misma lógica de gestión de memoria se extiende y se vuelve multidimensional para resolver problemas de organización de datos más complejos.

3. Arreglos Bidimensionales y Representación en Memoria

Las matrices son abstracciones fundamentales para datos tabulares. Sin embargo, un arquitecto debe recordar que la **RAM** no posee **"filas y columnas";** es un vector lineal. El éxito del procesamiento **2D** depende de cómo "aplanamos" esta estructura visual en una realidad física contigua.

3.1 Modelado de Matriz de Ocupación

Matriz salon **(3x4)** que representa puestos:

	Col 0	Col 1	Col 2	Col 3
Fila 0	[0][0]	[0][1]	[0][2]	[0][3]
Fila 1	[1][0]	[1][1]	[1][2]	[1][3]
Fila 2	[2][0]	[2][1]	[2][2]	[2][3]

- **Acceso (Pseudocódigo):** ESTADO puesto = salon[2][3]
- **Capacidad Total:** Se calcula como Filas \times Columnas = 3 \times 4 = 12 celdas.

3.2 Mapeo de Memoria (Row-Major Order)

Para persistir la matriz en RAM, utilizamos el orden por filas (Row-Major), visualizando la matriz de forma lineal y contigua:

[0,0][0,1][0,2][0,3] | [1,0][1,1][1,2][1,3] | [2,0][2,1][2,2][2,3]

Utilizando base 0x1000, 4 bytes por valor y 4 columnas, aplicamos: Dirección = Base + ((Fila * NumColumnas + Columna) * Tamaño)

- **[0][0]:** 0x1000 + ((0 \times 4 + 0) \times 4) = \mathbf{0x1000}

- **[1][2]:** 0x1000 + ((1 \times 4 + 2) \times 4) = 0x1000 + (6 \times 4) = 0x1000 + 24. En hex, 24 = 0x18. Resultado: **0x1018**.
- **[2][3]:** 0x1000 + ((2 \times 4 + 3) \times 4) = 0x1000 + (11 \times 4) = 0x1000 + 44. En hex, 44 = 0x2C. Resultado: **0x102C**.

En este cálculo, el **número de columnas** es la variable crítica, pues define el "paso" o salto necesario para navegar entre filas en un espacio unidimensional.

3.3 Procesamiento con Ciclos Anidados

Algoritmo formal para conteo de ocupación:

```
ENTERO contador = 0
FOR i FROM 0 TO filas - 1:
  FOR j FROM 0 TO columnas - 1:
    IF salon[i][j] == 1 THEN:
      contador = contador + 1
RETURN contador
```

La complejidad resultante es **O(f \times c)** o **O(n^2)** si la matriz es cuadrada), ya que el algoritmo debe visitar cada celda exactamente una vez. Si ambas dimensiones se duplican, el esfuerzo de procesamiento no se duplica, sino que se **cuadruplica** (2f \times 2c = 4fc), un impacto geométrico que debe preverse en el diseño de algoritmos sobre matrices extensas. Esta comprensión de cómo las dimensiones afectan el esfuerzo nos prepara para la toma de decisiones técnicas definitivas.

4. Síntesis y Conclusión: La Tríada del Rendimiento

La interacción entre Big O, los arreglos **1D** y las estructuras **2D** constituye la columna vertebral del pensamiento algorítmico profesional. No se trata simplemente de almacenar información, sino de estructurarla de modo que las operaciones críticas sean lo más eficientes posible según el hardware subyacente.

4.1 Conectando Conceptos

La complejidad **Big O** actúa como la brújula que determina la viabilidad de una estructura: mientras que un arreglo **1D** es óptimo para búsquedas binarias de alta velocidad **O(log n)** las matrices **2D** ofrecen una organización superior para datos

multidimensionales a cambio de un costo de procesamiento más elevado (O(n^2)). El conocimiento del mapeo en memoria permite al especialista optimizar estos recorridos, minimizando los saltos de caché y aprovechando la aritmética de punteros para lograr accesos **O(1**). En última instancia, la maestría en estructuras de datos reside en la capacidad de ofrecer una justificación técnica rigurosa que priorice la eficiencia y la escalabilidad sobre la mera sintaxis funcional.